

Session 08 — Homework

Starting point

Start from today's official Session End (what you built live plus your completed teamwork edit flow). Look closely at `AddTaskForm`'s `handleSubmit` — it already refuses to add a task when the trimmed title is empty:

```
if (trimmedTitle === "") {  
  return;  
}
```

That guard already exists and already works. The problem is that it fails **silently** — the form just does nothing, with no explanation. Your homework is to give that existing behavior (and a bit more) real, visible feedback.

CORE (required)

Goal

Give the existing empty-title rejection a visible error message. The rejection itself is not new — you are not building it from scratch, you are making it explain itself.

What to build

1. Give `AddTaskForm` a way to remember an error message ("Title can't be empty.") using `useState`.
2. When the existing empty-title guard runs (the `if (trimmedTitle === "")` check that's already there), set that error message instead of just silently returning.
3. Show the error on the page, near the form, only while it is set.
4. Once a task is successfully added, clear any previous error message.

Provided styling — copy as-is

CSS is not the exercise this session. Add this exactly as written to `src/index.css` — do not spend time redesigning it:

```
.form-error {  
  color: #c0392b;  
  font-size: 13px;
```

```
margin-top: 8px;
width: 100%;
}
```

Rules

Allowed/already known: `useState`, `.trim()` (already used by the existing guard), controlled inputs, conditional rendering (`condition && <p>...</p>` or a ternary). Not yet: `useEffect`, any Session 09+ concept.

Test it

- Submit the form with the input empty — an error appears, no task is added (the "no task is added" part already worked before today; only the visible error is new).
- Submit the form with only spaces typed — same result.
- Type a real title and submit — the error (if it was showing) disappears, and the task is added.

Done when

- ☐ Submitting an empty/whitespace-only title shows an error message (it already added nothing — now it also explains why).
- ☐ A successful add clears any previous error.
- ☐ `npm run build` passes.

STRETCH (optional)

Goal

Add a genuinely new check: also stop titles that are unreasonably long. Unlike CORE, this rejection does not exist yet at all.

What to build

1. Pick a maximum length for a task title (for example, 200 characters).
2. Reuse the same error-message state you built for CORE to show a message when the trimmed title is longer than your maximum.
3. A title at exactly the maximum length should be accepted; one character over should not.

Test it

- Paste a title far longer than your maximum — error shown, nothing added.
- A title exactly at your maximum — accepted.

Done when

- ☐ Over-length titles are rejected with a clear message.
 - ☐ The boundary (exactly at the maximum) is accepted, not rejected.
 - ☐ `npm run build` passes.
-

CHALLENGE (optional, harder)

Goal

You built error feedback for the Add form. Now bring the same idea — and the same two checks — to editing a task's title.

Right now, if you save an edit with an empty or whitespace-only title, your teamwork's `handleSaveClick` just closes editing mode without saving anything — no error, no explanation, same silent-failure problem CORE just fixed for Add.

If you skipped STRETCH, implement the empty-title edit error only. If you completed STRETCH, apply the same maximum-length rule to editing too.

What to build

1. Give `TaskItem` its own local error state (`useState`), separate from `AddTaskForm` 's.
2. Before calling `onSaveEdit` , check the trimmed title the same two ways you already check it in `AddTaskForm` : empty, and (if you built STRETCH) over your maximum length.
3. If either check fails: show an error message **and stay in editing mode** — do not close it, and do not call `onSaveEdit` .
4. If both checks pass: call `onSaveEdit` exactly as before, close editing mode, and clear the error.
5. Reuse the exact same `.form-error` CSS class from CORE — no new styling needed.
6. Make sure a fresh click on Edit (on any task) starts with a clean slate — no leftover error message from a previous edit attempt.

Rules

Allowed/already known: everything from CORE/STRETCH, applied inside `TaskItem` instead of `AddTaskForm` . Not yet: `useEffect` , keyboard event handling for saving/canceling, editing the task's owner, or any Session 09+ concept. You don't need any array method beyond what you've already used this session.

Test it

- Click Edit, clear the title completely, click Save — an error appears, the row stays in editing mode, the real task title is unchanged.
- Click Edit, type a valid new title, click Save — the title updates for real, editing mode closes, no error shown.
- Click Edit on a different task after seeing an error on a previous one — the new edit starts clean, no stale error message.
- Click Cancel while an error is showing — editing mode closes and the error goes away.

Done when

- ☐ An invalid edit (empty, or over your STRETCH maximum) shows an error and does NOT save or close editing mode.
- ☐ A valid edit still saves and closes editing mode exactly as it did after teamwork.
- ☐ Starting a new edit never shows a leftover error from a previous attempt.
- ☐ `npm run build` passes.